

NUnit Hands-On - ConverterLib (Moq)

1. Objective

Write NUnit test cases for the USDToEuro method of the Converter class in ConverterLib, using the Moq framework to isolate the dependency on the IDollarToEuroExchangeRateFeed service.

2. Solution Structure

Solution: ConverterLib.sln

Project 1 (given): ConverterLib -> Converter.cs, IDollarToEuroExchangeRateFeed.cs

Project 2 (created): ConverterLib.Tests -> ConverterTests.cs

References added to ConverterLib.Tests: ConverterLib (project reference), NUnit, NUnit3TestAdapter, Moq (via NuGet Package Manager).

3. Source Project Code

IDollarToEuroExchangeRateFeed.cs

```
namespace ConverterLib
{
    public interface IDollarToEuroExchangeRateFeed
    {
        double GetRate();
    }
}
```

Converter.cs

```
using System;

namespace ConverterLib
{
    public class Converter
    {
        private readonly IDollarToEuroExchangeRateFeed exchangeRateFeed;

        public Converter(IDollarToEuroExchangeRateFeed exchangeRateFeed)
        {
            this.exchangeRateFeed = exchangeRateFeed;
        }

        public double USDToEuro(double dollar)
        {
            if (dollar < 0)
            {
                throw new ArgumentException("Dollar amount cannot be negative");
            }

            double rate = exchangeRateFeed.GetRate();
            return dollar * rate;
        }
    }
}
```

4. Test Project Code - ConverterTests.cs

```
using System;
using Moq;
using NUnit.Framework;
using ConverterLib;

namespace ConverterLib.Tests
{
    public class ConverterTests
    {
        private Mock<IDollarToEuroExchangeRateFeed> mockFeed;
        private Converter converter;

        [SetUp]
        public void Setup()
        {
            mockFeed = new Mock<IDollarToEuroExchangeRateFeed>();
            converter = new Converter(mockFeed.Object);
        }

        [Test]
        public void USDToEuro_ValidDollarAmount_ReturnsConvertedEuroValue()
        {
            mockFeed.Setup(feed => feed.GetRate()).Returns(0.85);

            var result = converter.USDToEuro(100);

            Assert.That(result, Is.EqualTo(85));
        }

        [Test]
        public void USDToEuro_ZeroDollarAmount_ReturnsZero()
        {
            mockFeed.Setup(feed => feed.GetRate()).Returns(0.85);

            var result = converter.USDToEuro(0);

            Assert.That(result, Is.EqualTo(0));
        }

        [Test]
        public void USDToEuro_NegativeDollarAmount_ThrowsArgumentException()
        {
            mockFeed.Setup(feed => feed.GetRate()).Returns(0.85);

            Assert.That(() => converter.USDToEuro(-50),
                Throws.TypeOf<ArgumentException>());
        }

        [Test]
        public void USDToEuro_ValidDollarAmount_CallsExchangeRateFeedOnce()
        {
            mockFeed.Setup(feed => feed.GetRate()).Returns(0.9);

            converter.USDToEuro(50);

            mockFeed.Verify(feed => feed.GetRate(), Times.Once);
        }
    }
}
```

```
}
```

5. First Run - Test Result

Command used: dotnet test (or Test > Run All Tests in Visual Studio Test Explorer)

```
Test run for ConverterLib.Tests.dll (.NETCoreApp,Version=v6.0)
NUnit Adapter 4.5.0.0: Test execution started
Running all tests in ConverterLib.Tests.dll
  NUnit3TestExecutor discovered 4 of 4 NUnit test cases

Passed!  USDToEuro_ValidDollarAmount_ReturnsConvertedEuroValue
Passed!  USDToEuro_ZeroDollarAmount_ReturnsZero
Passed!  USDToEuro_NegativeDollarAmount_ThrowsArgumentException
Passed!  USDToEuro_ValidDollarAmount_CallsExchangeRateFeedOnce

Passed! - Failed:      0, Passed:      4, Skipped:      0, Total:      4,
Duration: 52 ms
```

6. Breaking the Test - Modified Source Code

USDToEuro in Converter.cs is modified to use a hardcoded rate instead of calling the exchange rate feed:

```
public double USDToEuro(double dollar)
{
    if (dollar < 0)
    {
        throw new ArgumentException("Dollar amount cannot be negative");
    }

    // double rate = exchangeRateFeed.GetRate();
    double rate = 1; // hardcoded, feed no longer called
    return dollar * rate;
}
```

7. Rerun - Test Result After Breaking Source

```
Running all tests in ConverterLib.Tests.dll
  NUnit3TestExecutor discovered 4 of 4 NUnit test cases

Failed!  USDToEuro_ValidDollarAmount_ReturnsConvertedEuroValue
  Message:
    Expected: 85
    But was: 100
Passed!  USDToEuro_ZeroDollarAmount_ReturnsZero
Passed!  USDToEuro_NegativeDollarAmount_ThrowsArgumentException
Failed!  USDToEuro_ValidDollarAmount_CallsExchangeRateFeedOnce
  Message:
    Mock.Verify failed: feed.GetRate() was not invoked.
    Expected invocation on the mock once, but was 0 times

Failed! - Failed:      2, Passed:      2, Skipped:      0, Total:      4,
Duration: 44 ms
```

8. Observation

Once the exchange rate feed is bypassed with a hardcoded rate, two tests fail: the conversion test (since 100 USD now returns 100 instead of the expected 85 at a 0.85 rate) and the Moq Verify test (since `GetRate()` is never invoked). This confirms Moq is correctly isolating and verifying the dependency on `IDollarToEuroExchangeRateFeed`. Restoring the call to `exchangeRateFeed.GetRate()` makes all 4 tests pass again.